

CSE 232 : Data Structures

LAB 03 : STACK

Due Saturday at 5:00 p.m. MST

The third programming assignment of this semester is to write a class with the same functionality as the standard template library (STL) `std::stack` class. As with all remaining labs this semester, this will be accomplished using the pair programming technique.

Stack

Your vector class will need to be defined in its own header file (`stack.h`). The class name must be `stack` (or, more accurately, `custom::stack`). It will need to support the following operations:

- **Construct:** default constructor, copy constructor, and move constructor.
- **Assign:** copy assign, move assign, and swap.
- **Access:** top (both read/write only).
- **Insert:** copy push and move push.
- **Remove:** pop.
- **Status:** size and empty.

Three Options

As with last week, you will have three options when implementing this class.

85 Option : Integer Base Type

The 85 option will require you to implement all the above interfaces, but integer will be the base data-type and the standard template library vector (`std::vector`) as the underlying data structure. There is a single member variable:

```
std::vector<int> container;
```

To complete the 85 option, please start with the following files: [232.03.Lab.85.zip](#).

100 Option : Template Base Type

The 100 option will require you to implement all the same interfaces, but a template will be the base type.

```
template <class T>
```

Use the standard template library vector (`std::vector`) as the underlying data structure.

```
std::vector<T> container;
```

To complete the 100 option, please start with the following files: [232.03.Lab.100.zip](#).

115 Option : Container

The 115 option will require the class to everything from the 100 option. Additionally, the client can specify an arbitrary container as the underlying data structure through a template parameter.

```
template<class T, class Container = custom::vector<T>>
```

The default data structure will be your vector (`custom::vector`) from last week. This requires you to get your "115" vector working and implement the following additional interfaces: vector's move assignment operator and vector's swap method. There is just one member variable:

```
Container container;
```

As with last week, the 115 option requires a precise number of allocations and deletions. In order to have more control over this, it is necessary to use an allocator. Please see Appendix I: Allocator in the text book for a detailed description of how this works. There is also a video demonstration here: [Allocators and Spys](#). This explains how the `SpY` class works to count allocations, and how `std::allocator` can be use to more precisely control them.

To complete the 115 option, please start with the following files: [232.03.Lab.115.zip](#).

Unit Tests

You will also be required to implement your own unit test for `stack::pop`. This will require you to add several functions in `testStack.h`.

Please create a video demonstrating the execution of your code. The video must be very short. No video longer than 2 minutes will be accepted. Demonstrate your code compiling and running the unit tests. Also, show your unit test code in the video and explain why you feel they adequately exercise `std::pop()`.

Please make sure you have your face in the video at some point in time. The easiest way to do this is to record the video with "picture-in-picture" turned on. Finally, a link to the video must be pasted into the comments section of the submission (do not attach the video file). Most students post their video on YouTube or record to the cloud with Zoom.

Assignment

Please do the following:

1. Select an option: 85, 100, or 115.
2. Download the code.
3. Implement the data structure with your partner(s).
4. Demonstrate your code, including compilation and execution. Upload to a streaming site and put a link in the comments section of the submission.
5. Upload your `stack.h` code.

Assessment

The grade for your assignment will be based on the following criteria:

Exceptional	Good	Acceptable	Developing	Missing
100%	90%	70%	50%	0%

Code Quality 35%	The most elegant solution was found for all the methods; there is no room for improvement,	There are no errors in efficiency, maintainability, cohesion, robustness, or redundancy.	There exists one error in efficiency, maintainability, cohesion, robustness, or redundancy.	The code quality is not what is expected for a CSE 232 student.	The code is sloppy or otherwise unprofessional.
Functionality 50%	Passes all the tests	Passes 90% of unit tests	Passes 70% of unit tests	Passes 50% of unit tests	Program fails to compile or no demonstration video was provided.
Template Base Type 15%	The 100 or the 115 solution was attempted				The 85% solution was attempted
Container 15%	115 solution was attempted				The 85 or the 100 solution was attempted